ALGOINVEST&TRADE

MARS 2026

OPTIMISATION D'UNE STRATÉGIE D'INVESTISSEMENT AVEC DES ALGORITHMES PYTHON

PRESENTÉ PAR
Réda Abdi

POUR
AlgoInvest&Trade

ALGOINVEST&TRADE

SOMMAIRE

I - Introduction
a. La problématique 3
b. Le problème du sac à dos

II - Solution brute-force
a. L'algorithme 6
b. Limites et transition vers une solution plus optimisé

III - Solution optimisé
a. La programation dynamique 8
b. L'algorithme dynamique en pseudocode

IV - Comparaison d'efficacité et de performances, avec Big-O
a. Définition notation Big-O 16
b. Brute-force vs version optimisé (programmation dynamique)

V - Algorithme vs humain : Comparaison sur des données antérieurs 19
a. Présentation des données
b. Rapports d'exploration

I - INTRODUCTION

a. La problématique

- **Contexte :** La société d'investissement AlgoInvest&Trade souhaitent optimiser leur stratégie d'investissement à l'aide d'algorithmes afin de maximiser le profit de ses clients. Une stratégie d'investissement est une sélections d'actions choisies parmi les actions disponibles, dont le coût total respecte le budget imparti.

- **Objectif** : Créer un algorithme qui renvoie la sélection d'actions la plus rentable après deux ans d'investissement, sous le budget imparti.

- **Contraintes :**
  - Chaque action ne peut être acheté qu'une fois
  - L'action ne peut pas être acheté en fraction
  - Le budget est de 500 euros par client

I - INTRODUCTION

a. La problématique

- **Exemple d'une liste d'action :**
  Chaque action à un nom, un poids (coût) et une valeur (bénéfice après 2 ans)

Actions #	Coût par action (en euros)	Bénéfice (après 2 ans)
Action-1	20	5%
Action-2	30	10%
Action-3	50	15%
Action-4	70	20%
Action-5	60	17%
Action-6	80	25%
Action-7	22	7%
Action-8	26	11%
Action-9	48	13%
Action-10	34	27%
Action-11	42	17%
Action-12	110	9%
Action-13	38	23%
Action-14	14	1%
Action-15	18	3%
Action-16	8	8%
Action-17	4	12%
Action-18	10	14%
Action-19	24	21%
Action-20	114	18%

I - INTRODUCTION

b. Le problème du sac à dos

- **Définissons le problème :**

On se retrouve dans un problème classique en algorithmique : le problème du sac à dos (Knapsack problem) . C’est un problème d'optimisation combinatoire, qui prend en compte un contenant qui peut contenir un poids maximum (le budget de 500 euros dans notre cas), et une liste d’objets qui pèsent un poids et ont une valeur (la liste d’actions). Le but est de trouver de manière efficace la combinaison d’objets qui apportera le plus de valeur total, tout en respectant le poids maximum du contenant.

- **Comment résoudre ?**

L’approche de résolution la plus instinctive est celle qui consiste à essayer toutes les combinaisons d’actions, et à garder la séléction la plus rentable sous la contrainte budgetaire. C’est ce qu’on appelle : la solution brute-force .

II - SOLUTION BRUTE-FORCE

a. L'algorithme

- **Principe:** La solution brute-force essaie toutes les combinaisons d'actions possibles, et lorsque la combinaison respecte le budget, elle la compare avec la combinaison la plus rentable des dernières essayés, et stocke la plus rentable d'entre les deux.

- **Algorithme en pseudocode :** Ci-dessous une représentation en pseudo-code de la fonction que j'ai élaboré pour optimiser nos stratégies d'investissement, en utilisant un algorithme brute-force. La fonction prend en paramètre une variable "liste_actions" qui est une liste d'instances de la classe Action.

```
FONCTION brute_force(liste_actions):
  meilleure_combinaison = {combinaison_actions = [ ], cout_total = 0, benefice_total = 0}
  POUR combinaison_possible (liste_actions) :
    cout_total = 0
    benefice_total = 0
    POUR action DANS combinaison_possible:
      cout_total += action.cout
      benefice_total += action.benefice_euros
    SI cout_total <= 500 ET benefice_total > meilleure_combinaison["benefice_total"] :
      meilleure_combinaison["combinaison_actions"] = combinaison_possible
      meilleure_combinaison["cout_total"] = cout_total
      meilleure_combinaison["benefice_total"] = benefice_total

  RETOURNE meilleure_combinaison
```

II - SOLUTION BRUTE-FORCE

b. Limites et transition vers une solution optimisé

- **Limites :**

L'algorithme est efficace pour une courte portion d'action, mais le temps d'exécution devient trop long lorsque le nombre d'actions augmente. Ce qui nous amène à réfléchir à une solution plus optimisé qui permettra de calculer un nombre important d'actions.

- **Comment optimiser ?**

Il existe differentes approche de résolution pour le problème du sac à dos, les plus utilisés sont :

- L'approche gloutonne : Avantage → Rapide
Inconvenient → Renvoie parfois une solution proche de la meilleure solution

- La programmation dynamique : Avantage → Précis
Inconvénient → Coûteux en ressources (temps & mémoire) mais beaucoup moins que la brute-force

Nous allons privilégier la programmation dynamique pour notre cas.

III - SOLUTION OPTIMISÉ

a. La programation dynamique

- **Qu’est ce que la programmation dynamique ?**

La programmation dynamique est une méthode utilisée pour résoudre des problèmes algorithmiques complexes en les décomposant en plusieurs sous-problèmes plus petits. Elle est particulièrement utile lorsque ces sous-problèmes se chevauchent (c’est-à-dire que les mêmes sous-problèmes demandent d’être résolu plusieurs fois au cours de la résolution), et en stockant leur résultats afin d’éviter de les recalculer (et donc d’économiser de la puissance de calcul).

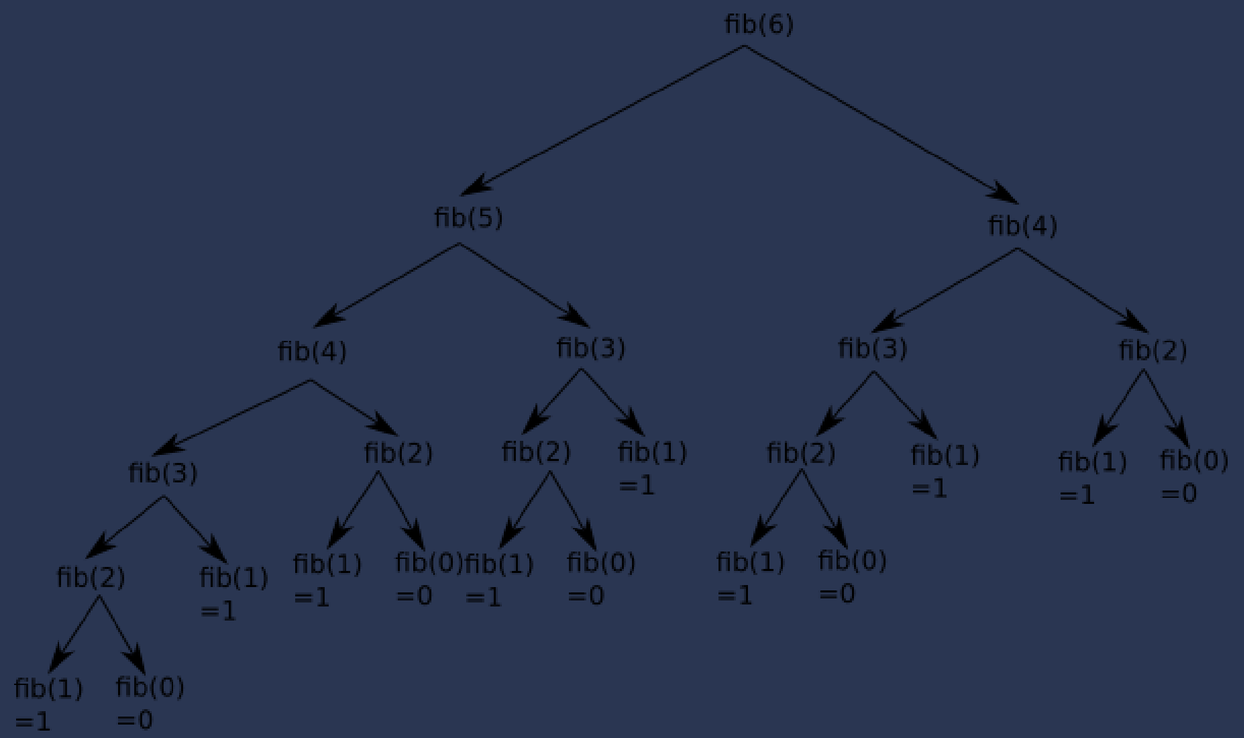
- **Exemple :**

Commençons par un exemple simple en utilisant la programmation dynamique 1D (à 1 dimension) : la suite de Fibonacci.
Rappelons son principe : fib(0) est égal à 0 et fib(1) est égal à 1. Pour calculer les termes suivants de la suite, il suffit d’additionner les deux termes précédents. Par exemple fib(2) est égal à 1 et fib(3) est égal à 2.

III - SOLUTION OPTIMISÉ

a. La programation dynamique

Comme vous pouvez le voir avec le schéma ci-dessous, on peut très vite se retrouver à calculer les mêmes termes plusieurs fois si on ne sauvegarde pas les précédents résultats. C'est là que la programmation dynamique intervient. Nous allons donc créer un algorithme qui, au lieu de calculer chaque terme un par un, sauvegardera les sous-problèmes qui se chevauchent, dans une liste.



III - SOLUTION OPTIMISÉ

a. La programation dynamique

- **Fonction Fibonacci qui calcul et sauvegarde les termes Fibonnaci du plus petit jusqu'à trouver n .**

```
FONCTION Fibonnaci(n) :
  dp = [0,1]
  Si n < 2 :
    RETOURNE dp[n]
  Sinon :
    POUR i allant de 2 à n+1 :
      dp.ajouter(dp[i-1] + dp[i-2])

  RETOURNE dp[n]
```

*Remarque : Les résultats sont stockés dans une liste, c'est pour cela que nous parlons de programmation dynamique à 1 dimension.

III - SOLUTION OPTIMISÉ

a. La programation dynamique

- **Solution optimisé en pseudocode**

C’est donc ce principe de programmation dynamique que nous allons utiliser afin de développer une solution efficace pour maximiser les bénéfices de nos clients. Cette fois-ci la matrice sera en 2 dimensions et la fonction principale se décomposera en 3 parties :

- Création de la matrice
- Remplissage de la matrice
- Backtracking : Assigner la liste d’actions correspondant au bénéfices attribué à la dernière case

III - SOLUTION OPTIMISÉ

b. L'algorithme dynamique en pseudocode

- **Fonction**

La fonction prend en paramètre une liste d'action matéralisé sous formes d'instances de classe, ainsi que un budget maximum (précision : ce budget doit être entré sous forme d'unité, c'est à dire que 500€ devra plûtot être rentré sous forme de centimes)

- **Création de la matrice**

La matrice à 2 dimensions est un tableau : chaque colonne représente un budget, entre 0 et le budget max imposé, et chaque ligne représente une action (la premère ligne doit représenter aucune action). Ainsi chaque case représentera le bénéfice maximum que l'on peut tirer sous le budget et la séléction d'actions imparti.

```
FONCTION programmation_dynamique (liste_actions, budget_max) :

  #Création de la matrice vide
  matrice = []
  POUR i allant à taille(liste_actions) + 1 :
      ligne = []
      POUR j allant à budget_max + 1 :
          ligne.ajouter(0)
      matrice.ajouter(ligne)
```

III - SOLUTION OPTIMISÉ

b. L'algorithme dynamique en pseudocode

- **Remplissage de la matrice**

Une fois la matrice créée nous allons procéder de cette manière :
Nous allons pour chaque ligne, calculer le bénéfice optimale dans le budget imparti (colonne) en incluant ou non l'action (ligne).
Pour cela nous allons, pour chaque case, comparer le bénéfice optimale entre :

- L'action inclut (si coût rentre dans le budget) : bénéfice de l'action + bénéfice de la case qui exclut l'action (donc la ligne au dessus) et qui a pour budget le budget actuel - le coût de l'action
- L'action exclut : La ligne précédente

```
#Remplissage de la matrice
POUR action_index allant de 1 à taille(liste_actions) + 1 :
  action = liste_actions[action_index - 1]
  POUR budget allant de 1 à budget_max + 1 :
    SI action.coût <= budget :
      matrice[action_index][budget] = maximum(matrice[action_index - 1][budget],
      action.benefices + matrice[action_index - 1][budget-action.coût])
    SINON :
      matrice[action_index][budget] = matrice[action_index - 1][budget]
```

III - SOLUTION OPTIMISÉ

b. L’algorithme dynamique en pseudocode

- **Backtracking**

Faire du backtracking afin de retrouver les actions correspondantes au résultat final est moins coûteux en ressources que d’assigner une liste d’action à chaque case. Pour cela, une fois la matrice remplie, nous allons faire le chemin en sens inverse. Nous allons partir de la dernière case (donc séléction d’actions maximum et budget maximum) et comparer, dans le même budget, la ligne précédentes (donc en excluant la dernière action),

- Si le bénéfice est le même, cela signifie que l’action n’a pas été attribué.
- Si le bénéfice n’est pas le même, (donc correspond au bénéfice de l’action additionné au bénéfice de la case excluant l’action et ayant pour budget le budget actuel - le côut de l’action) alors l’action a été séléctionné.

Nous passons ensuite à la ligne d’au dessus, et si l’action à été séléctionné, nous passons alors également à la colonne ayant pour budget le coût de l’action soustrait au budget actuel.

```
#Retro engineering : Assigner une liste d'actions à la dernière case
liste_actions_optimale = []
budget_actuel = budget_max
index_action = taille(liste_actions)
TANT QUE budget_actuel > 0 ET index_action > 0 :
  action = liste_actions[index_action - 1]
  SI matrice[index_action][budget_actuel] == action.benefices + matrice[index_action-1]
  [budget_actuel - action.cout] :
    liste_actions_optimale.ajouter(action)
    budget_actuel -= action.cout
  index_action -= 1
```

III - SOLUTION OPTIMISÉ

b. L’algorithme dynamique en pseudocode

- **Cas limites**

1) Cette algorithme est cohérent dans la limite des conditions imposés, mais ne prend pas en compte la possibilité d’ajout de nouvelles contraintes plus complexes.

2) Il à également été conçu dans le cadre d’un problème algorithmique knapsack 0/1. C’est à dire avec l’impossibilité de fractionner les actions afin d’en séléctionner la quantité souhaité ou de par exemple acheter une action plusieurs fois.

3)Enfin, malgré son efficacité, l’algorithme reste coûteux en ressources, lorsque la data grimpe en nombre, le temps d’execution peut devenir long. Détaillons cela en comparant, et en utilisant la notation Big-O.

IV - COMPARAISON EFFICACITÉ ET PERFORMANCES AVEC BIG-O

a. Définition notation Big-O

- **Définition**

La notation Big O (ou grand O) est une notation mathématique utilisée en informatique pour décrire la complexité asymptotique d'un algorithme, c'est-à-dire la façon dont son temps d'exécution ou son utilisation mémoire évolue lorsque la taille de l'entrée augmente.

C'est une méthode qui permet de mesurer simplement la performance d'un algorithme. Sans prendre en compte les caractéristiques de la machine(langage, processeur, etc.), mais simplement en observant comment le nombre d'opérations ou la quantité de mémoire utilisée évolue.

IV - COMPARAISON EFFICACITÉ ET PERFORMANCES AVEC BIG-O

b. Brute-force vs version optimisé (programmation dynamique)

- **Comparaison de la complexité temporelle avec Big-O**

Force-brute

Pour n = taille de liste_actions

2^n

n

```
FONCTION brute_force(liste_actions):
  meilleure_combinaison = {combinaison_actions = [ ],
  cout_total = 0,benefice_total = 0}
  POUR combinaison_possible (liste_actions) :
    cout_total = 0
    benefice_total = 0
    POUR action DANS combinaison_possible:
      cout_total += action.cout
      benefice_total += action.benefice_euros
    SI cout_total <= 500 ET benefice_total > meilleure_combinaison
    ["benefice_total"] :
      meilleure_combinaison["combinaison_actions"] = combinaison_po
      ssible
      meilleure_combinaison["cout_total"] = cout_total
      meilleure_combinaison["benefice_total"] = benefice_total

  RETOURNE meilleure_combinaison
```

Complexité temporelle = O(n x 2^n)

Version optimisé (programmation dynamique)

Pour n = taille de liste_actions
Pour W = budget

n

W

n

W

n

```
FONCTION programmation_dynamique (liste_actions, budget_max) :

  #Création de la matrice vide
  matrice = []
  POUR i allant à taille(liste_actions) + 1 :
    ligne = []
    POUR j allant à budget_max + 1 :
      ligne.ajouter(0)
    matrice.ajouter(ligne)

  #Remplissage de la matrice
  POUR action_index allant de 1 à taille(liste_actions) + 1 :
   action = liste_actions[action_index - 1]
   POUR budget allant de 1 à budget_max + 1 :
     SI action.coût <= budget :
       matrice[action_index][budget] = maximum(matrice[acti
       on_index - 1][budget],action.benefices + matrice[act
       ion_index - 1][budget-action.coût])
     SINON :
       matrice[action_index][budget] = matrice[action_inde
       x - 1][budget]

  #Backtracking : Assigner une liste d'actions à la dernière case
  liste_actions_optimale = []
  budget_actuel = budget_max
  index_action = taille(liste_actions)
  TANT QUE budget_actuel > 0 ET index_action > 0 :
    action = liste_actions[index_action - 1]
    SI matrice[index_action][budget_actuel] == action.benefices +
    matrice[index_action-1]
    [budget_actuel - action.cout] :
      liste_actions_optimale.ajouter(action)
      budget_actuel -= action.cout
    index_action -= 1
```

Complexité temporelle de O(n x W + n x W + n)
soit ≃ O(n x W)

IV - COMPARAISON EFFICACITÉ ET PERFORMANCES AVEC BIG-O

b. Brute-force vs version optimisé (programmation dynamique)

- **Comparaison de la complexité temporelle avec Big-O**

On peut en déduire que la solution brute_force à une complexité temporelle exponentielle et donc trop coûteuse lorsque le nombre d'actions augmente significativement. La solution optimisé possède quant à elle un temps d'exécution polynomiale. Malgré le fait que l'on utilise majoritairement la notation Big-O pour mesurer la complexité temporelle, nous pouvons également l'utiliser afin d'analyser l'utilisation mémoire de la fonction.

- **Analyse de l'utilisation mémoire avec Big-O**

Lorsque nous calculons la complexité d'espace d'une fonction, nous ne prenons pas en compte l'entrée de mémoire, mais seulement la quantité de mémoire qui va devoir être créer selon les paramètres de la fonction.

La solution brute-force par exemple possède quelques variables mais qui reste constantes donc d'une complexité O(1), ainsi qu'une variable "combinaisons_possibles" qui est à chaque fois renouvelé et parfois stocké, d'une taille maximum de n, donc d'une complexité O(n). On peut donc noter que la complexité totale de mémoire de la solution brute-force se note $\simeq$ O(n) .

La version programmation dynamique quant à elle possède aussi quelque variable de taille constantes donc O(1), cependant elle stocke une matrice de taille n x W. Donc la complexité de cette solution se note $\simeq$ O(n).

IV - COMPARAISON EFFICACITÉ ET PERFORMANCES AVEC BIG-O

b. Brute-force vs version optimisé (programmation dynamique)

- **Conclusion**

En résumé, la programmation dynamique améliore fortement la complexité temporelle par rapport à la force brute, au prix d'une utilisation plus importante de la mémoire. Cette approche devient particulièrement efficace lorsque la taille des données reste compatible avec les ressources de la machine. Elle représente donc un compromis entre temps de calcul et consommation mémoire.

V - ALGORITHME VS HUMAIN : COMPARAISON SUR DES DONNÉES ANTÉRIEURS

a. Présentation des données

- **Contexte**

Des données antérieurs m'ont été transmis par Sienna, ainsi que les choix d'investissments qui ont été fait manuellement à cette époque. C'est une bonne opportunité de montrer l'impact de l'automatisation et d'optimisation des choix d'invesstissment avec notre algorithme, mais également, au vu de la quantité de données, de tirer des informations intérressantes au travers d'un rapport d'exploration des données .

Voici un présentation des fichiers que nous allons analyser :

- dataset1 & 2 : Chaque fichier représente des centaines d'actions utilisé par l'entreprise il y'a quelques années. La variable price correspond au coût en euro de l'action représenté (elle ne peut être acheté qu'une fois et ne peut être fractionné) tandis que la variable profit représente le bénéfice en euro de l'action en question.

V - ALGORITHME VS HUMAIN : COMPARAISON SUR DES DONNÉES ANTÉRIEURS

a. Présentation des données

- **Contexte**

Extrait dataset1.csv

1	name	price	profit
2	Share-DUPH	10.01	12.25
3	Share-GTAN	26.04	38.06
4	Share-USUF	9.25	27.69
5	Share-CFOZ	10.64	38.21
6	Share-QLRX	15.72	27.47
7	Share-XJDT	0.0	37.11
8	Share-HKFP	23.97	19.66
9	Share-PPPH	24.06	38.2
10	Share-HLJY	26.98	5.54

Extrait dataset2.csv

1	name	price	profit
2	Share-MOEX	40.6	16.69
3	Share-GBGY	27.08	34.09
4	Share-AXWK	-9.27	27.19
5	Share-FJTI	33.5	20.81
6	Share-LGDP	15.26	3.4
7	Share-GEBJ	5.87	37.95
8	Share-GGNT	33.39	7.88
9	Share-FDLS	-6.38	10.96
10	Share-JDYS	-2.7	11.96

V - ALGORITHME VS HUMAIN : COMPARAISON SUR DES DONNÉES ANTÉRIEURS

a. Présentation des données

- **Contexte**

- sienna_decision_achat_dataset1 & 2 : Correspond à la sélection d'actions que Sienna à faite pour chaque dataset, pour un budget de 500€ maximum

- algorithme_decision_achat_dataset1 & 2 : Correspond à la sélection d'actions que l'algorithme à fait pour chaque dataset, pour un budget de 500€ maximum

sienna_decision_achat_dataset1

```
Sienna bought:

Share-GRUT

Total cost: 498.76€
Total return: 196.61€
```

sienna_decision_achat_dataset2

```
Sienna bought:
Share-ECAQ 3166
Share-IXCI 2632
Share-FWBE 1830
Share-ZOFA 2532
Share-PLLK 1994
Share-YFVZ 2255
Share-ANFX 3854
Share-PATS 2770
Share-NDKR 3306
Share-ALIY 2908
Share-JWGF 4869
Share-JGTW 3529
Share-FAPS 3257
Share-VCAX 2742
Share-LFXB 1483
Share-DWSK 2949
Share-XQII 1342
Share-ROOM 1506

Total cost: 489.24€
Profit: 193.78€
```

algorithme_decision_achat_dataset1

```
> Share-QQTU :
  Cost : 33.19€
  Benefits after 2 years : 13.14€(39.6%)

> Share-GIAJ :
  Cost : 10.75€
  Benefits after 2 years : 4.29€(39.9%)

> Share-XJMO :
  Cost : 9.39€
  Benefits after 2 years : 3.75€(39.98%)

> Share-LRBZ :
  Cost : 32.9€
  Benefits after 2 years : 13.14€(39.95%)

> Share-KZBL :
  Cost : 28.99€
  Benefits after 2 years : 11.35€(39.14%)

> Share-EMOV :
  Cost : 8.89€
  Benefits after 2 years : 3.51€(39.52%)

> Share-IFCP :
  Cost : 29.23€
  Benefits after 2 years : 11.66€(39.88%)

Total cost : 499.96€
Total benefits (after 2 years) : 198.55€

Time : 12.16917824745178 2 seconds
```

algorithme_decision_achat_dataset2

```
> Share-JGTW :
  Cost : 35.29€
  Benefits after 2 years : 13.91€(39.43%)

> Share-FAPS :
  Cost : 32.57€
  Benefits after 2 years : 12.88€(39.54%)

> Share-VCAX :
  Cost : 27.42€
  Benefits after 2 years : 10.69€(38.99%)

> Share-LFXB :
  Cost : 14.83€
  Benefits after 2 years : 5.9€(39.79%)

> Share-DWSK :
  Cost : 29.49€
  Benefits after 2 years : 11.6€(39.35%)

> Share-XQII :
  Cost : 13.42€
  Benefits after 2 years : 5.3€(39.51%)

> Share-ROOM :
  Cost : 15.06€
  Benefits after 2 years : 5.91€(39.23%)

Total cost : 499.92€
Total benefits (after 2 years) : 197.96€

Time : 6.608839750289917 seconds
```

V - ALGORITHME VS HUMAIN : COMPARAISON SUR DES DONNÉES ANTÉRIEURS

b. Rapports d’exploration

- **Rapport d'exploration dataset 1 & 2**

	Nombre d’actions	Nombre d’actions erronés (0 ou négatif)	Coût min	Coût max	Coût moy	Coût median	Coût écart type	Bénefice min	Bénefice max	Bénefice moy	Bénefice median	Bénefice ecart-type
DATASET 1	1001	44	0.01€	498.76€	25.54€	25.62 €	19.37€	0.02%	39.98%	20.23%	19.82%	11.48%
DATASET 2	1000	459	0.12€	51.46€	25.61€	26.12€	10.50€	0.15%	39.97%	19.57%	19.71%	12.20%

V - ALGORITHME VS HUMAIN : COMPARAISON SUR DES DONNÉES ANTÉRIEURS

b. Rapports d'exploration

- **Rapport d'exploration choix de Sienna vs choix de l'algorithme**

DATASET1	Sienna	Algorithme
Coût total	498,76€	499,96€
Bénéfice brut	695,37€	698,51€
Bénéfice net	196,61€	198,55€
Bénéfice en %	+39,42%	+39,71%

Dataset 1 :
Ici le coût des choix de l'algorithme est certes 1€ plus élevé que ceux de Sienna, mais la différence avec les bénéfices net et brut couvrent largement cette somme. Le bénéfice en pourcentage est aussi plus élévé pour l'algorithme. Jusqu'ici tout semble cohérent.

V - ALGORITHME VS HUMAIN : COMPARAISON SUR DES DONNÉES ANTÉRIEURS

b. Rapports d'exploration

- **Rapport d'exploration choix de Sienna vs choix de l'algorithme**

Dataset 2 :
Là, on peut observer une différence significative de 10€ plus élevé pour le choix de la machine, avec un bénéfice net de seulement 4€ plus élevé que Sienna, et même un bénéfice en pourcentage moins élevé que celui de Sienna.
Les choix de la machine sont-ils donc moins optimaux que les choix de Sienna ?
C'est ce que on pourrai se dire à première vue et conclure que le choix humain est plus efficace.
Ce raisonnement semble intuitivement juste, mais est erroné dans le contexte de ce problème.

En réalité la bonne lecture est que le bénéfice net absolu est le seul juge.
L'objectif du problème est de maximiser la différence entre le coût et le bénéfice brut. Seulement.
Sachant que le coût total est déjà pris en compte dans le calcul du bénéfice net, il n'est pas utile de les comparer. Les autres données sont intérressante à titre de comparaisons relatives.

L'humain à subi ici un biais cognitif dont la machine, elle, ne fait pas face.
Ce n'est pas seulement plus optimal en termes de rapidité d'execution d'automatiser nos programmes d'investissements, c'est surtout plus optimal en termes de performance pure du produit que nous délivrons.

DATASET2	**Sienna**	Algorithme
Coût total	489,24€	499,92€
Bénéfice brut	683,02€	697,88€
Bénéfice net	193,78€	197,96
Bénéfice en %	+39,6%	+39,59%

ALGOINVEST&TRADE

MERCI DE VOTRE TEMPS

Email

redaabdi@algoinvestandtrade.com

Réseaux

linkedin.com/in/redaabdi/

Contact téléphonique

07 82 88 37 12